INTERNAL TRADING TOOL

Options Screener

Architecture & Deployment

A Streamlit web app for screening puts & LEAPS, tracking portfolio P&L, and monitoring daily net liquidating value across broker accounts.

Python 3.14 | Streamlit 1.65 | Supabase | GitHub + Streamlit Cloud

October 2026

ARCHITECTURE — OVERVIEW

What the App Does

01

Options Screener

- Scan watchlist for put-selling opportunities
- Screen LEAPS for long-dated call exposure
- Filter by delta, IV rank, days to expiry
- Watchlists saved and shared across sessions

02

Portfolio Tracker

- Live positions via Tastytrade REST API
- FIFO P&L matching across multiple accounts
- Realized vs. unrealized P&L breakdown
- Transaction history with lot-level detail

03

NLV History

- Daily net liquidating value snapshots
- Rolling chart per broker account
- Backfill from Tastytrade history API
- Persistent storage in Supabase Postgres

Six-Layer Architecture

Presentation	Streamlit multi-page app · st.session_state · on_change callbacks · Plotly charts	`app.py`
Screener Engines	Put screener · LEAPS screener · configurable watchlists · delta / IV rank / DTE filters	`screener/`
Data Layer	yfinance option chains · scipy Black-Scholes Greeks · pandas-market-calendars DTE	`data/`
Broker Integration	Tastytrade REST API · OAuth session management · positions, balances, transactions	`portfolio/`
Persistence	Supabase Postgres (primary) · local JSON fallback · dual-backend pattern per store	`*_store.py`
Deployment	GitHub private repo · Streamlit Community Cloud auto-deploy · viewer email auth	`cloud`

Code Structure

Root

app.py
Streamlit entry point, page routing

config.py
PUT_UNIVERSE, MAGNIFICENT_10 defaults

watchlist_store.py
Supabase-backed watchlist persistence

options_data.py
yfinance wrapper — chains + pricing

scoring.py
Black-Scholes Greeks, IV rank

technical_analysis.py
Moving averages, trend indicators

vix_data.py
VIX fetch for market context

market_movers.py
Top gainers / losers feed

tastytrade_client.py
Tastytrade auth + session management

ssl_patch.py
Windows certificate store fix (no-op on Linux)

screeners/

put_screener.py
Delta / IV / DTE filtering logic

leaps_screener.py
Long-dated call screening logic

spread_screener.py
Vertical spread candidates

portfolio/

tastytrade_positions.py
Live positions + balances fetch

pnl_calculator.py
FIFO P&L matching across lots

snapshot_store.py
Daily NLV persistence (Supabase + JSON)

etrade_client.py · schwab_client.py
Additional broker stubs

Screener Engines

Put Screener

- Scans a configurable watchlist (30+ large-cap defaults)
- Filters: delta ≤ 0.30, IV rank ≥ 30, DTE 20–45 days
- Ranks by premium yield and open interest
- Shows bid/ask/mid per strike with Greeks

```
save_watchlist("put", symbols)
```

Watchlist persists to Supabase on every edit via **on_change** callback

LEAPS Screener

- Targets 12–24 month expiry long calls
- Filters: delta 0.60–0.80, price above 200-day MA
- Shows cost basis and break-even price at expiry
- Tracks the Magnificent 10 universe by default

```
save_watchlist("leaps", symbols)
```

Same persistence mechanism — each list stored under its own key in Supabase

Data Layer

MKT

Market Data

`yfinance 1.3.0`

- OHLCV price history
- Live option chains (calls + puts)
- Underlying price + 200-day MA
- curl_cffi on Windows for WFP bypass

Δ

Greeks Engine

`scipy 1.17.1`

- Black-Scholes via scipy.stats.norm
- Delta, Theta, Vega per contract
- Implied vol via Brent's method
- IV rank calculated from 52-week range

CAL

Trading Calendar

`pandas-market-calendars`

- NYSE schedule + holidays
- Accurate DTE calculation
- Handles half-days and closures
- Used to filter non-trading-day expirations

Broker Integration

Tastytrade API

Authentication

- Username + password → session token
- Token cached in st.session_state
- Auto-refreshes on expiry

Data Fetched

- Account balances + NLV
- Current positions with cost basis
- Transaction history for P&L
- Historical NLV for backfill

```
https://api.tastyworks.com/accounts/{id}/balances
https://api.tastyworks.com/accounts/{id}/positions
https://api.tastyworks.com/accounts/{id}/transactions
```

Design Notes

- Multi-account support — one token per account key
- NLV snapshots saved daily on portfolio page load
- Backfill runs once per account via merge_api_history()
- All API calls wrapped in try/except — graceful degradation when offline
- Supports future broker additions via same interface

Portfolio & P&L

FIFO P&L Matching

- Transactions sorted by date ascending
- Each buy creates a lot: rem_qty + price_per_unit
- Sells consume oldest lots first, quantity-aware
- Realized P&L = (sell price − cost basis) × qty
- Handles partial fills and split lots correctly

```
portfolio/pnl.py → fifo_match(trades)
```

NLV Snapshots

- Portfolio page saves today's NLV on load
- One row per broker per day in Supabase
- Upsert — safe to reload multiple times
- Plotly line chart with per-broker traces
- Backfill endpoint fetches up to 2 years history

```
snapshot_store.save_today(broker_balances)
```

DEPLOYMENT — STEP 1 OF 3

GitHub: Source Control

Repository Setup

- Private repo on GitHub — options_screener/ as root
- Parent folder excluded (contains unrelated personal files)
- Branch: master — Streamlit Cloud deploys from here
- Push triggers automatic redeploy within ~30 seconds

Key .gitignore Rules

```
.streamlit/secrets.toml
portfolio/nlv_snapshots.json
watchlists.json
debug_portfolio.py
find_history_endpoint.py
migrate_to_db.py
```

Credentials and local data files never enter the repo. A **secrets.toml.example** documents the required structure.

DEPLOYMENT — STEP 2 OF 3

Supabase: Cloud Database

Table: nlv_snapshots

```
date TEXT NOT NULL,
broker TEXT NOT NULL,
value FLOAT NOT NULL,
PRIMARY KEY (date, broker)
```

One row per broker per day. Upsert on load — safe to reload. Backfill uses ignore_duplicates=True.

Table: watchlists

```
key TEXT PRIMARY KEY,
symbols TEXT NOT NULL
-- symbols stored as
JSON array
```

Keys: "put" and "leaps". Written on every watchlist edit. Read once at app startup to seed session state.

Dual-Backend Pattern

- Each store calls _get_client() first
- Reads st.secrets["supabase"] for URL + key
- Falls back silently to local JSON if no credentials
- Same interface — zero code changes for dev vs. prod

```
watchlist_store.py
portfolio/snapshot_store.py
```

DEPLOYMENT — STEP 3 OF 3

Streamlit Community Cloud

Deploy Steps

1. Go to **share.streamlit.io** and sign in with GitHub
2. New app → connect private GitHub repo → set main file to **app.py**
3. Advanced settings → Secrets panel → paste Supabase URL + key in TOML format
4. Deploy — app builds from **requirements.txt** automatically

Secrets Panel

```
[supabase]
url = "https://…supabase.co"
key = "sb_publishable_…"
```

Secrets are encrypted at rest, never stored in the repo, and injected as **st.secrets** at runtime.

Auto-Deploy Pipeline

`git push` → GitHub webhook → Live in ~30s

Access Control

Viewer Authentication

Streamlit Community Cloud's built-in viewer auth restricts access to a named list of email addresses — no custom login code required.

Setup steps:

- App settings → Sharing → "Only specific people can view this app"
- Enter each viewer's email address (Google / GitHub accounts)
- Visitors see a Google/GitHub sign-in prompt
- Only approved emails gain access to the app

Security Properties

- No passwords stored in the app
- Auth handled entirely by Streamlit + Google/GitHub OAuth
- Credentials never touch the repo or Supabase
- Easy to add or remove viewers without a redeploy

Approved viewers:

```
socialraj@gmail.com
+ any additional users
```

Local Development

Setup

1. Install dependencies

```
pip install -r requirements.txt
```

2. Create local secrets file

```
# .streamlit/secrets.toml (gitignored)
[supabase]
url = "https://…supabase.co"
key = "sb_publishable_…"
```

3. Run the app

```
streamlit run app.py
```

Without Supabase Credentials

The app falls back to local JSON files automatically. No secrets file needed for basic development.

- watchlists.json — stores screener watchlists
- portfolio/nlv_snapshots.json — stores NLV history

One-Time Migration

To push existing local data to Supabase:

```
python migrate_to_db.py
```

Reads both JSON files and bulk-upserts to Supabase in 500-row batches. Run once, then delete.